SER335 Lab 1 ONLINE: Practice Linux, VMs, and Docker

Objectives:

- Navigate the National Vulnerabilities Database
- Apply C-I-A to the real world
- Demonstrate a working competency in Linux, specifically the bash shell
- Demonstrate a working competency in Docker

Submission:

- Submit a pdf file named "<asurite>_ser335_lab1.pdf" to Canvas that contains your answers to Task 1 and your screenshots Tasks 2, 3 and 4. Take screen captures as indicated in the task scripts below in the yellow callout boxes. Label them according to the number given.

Background:

Basic competency in Linux (and ideally, all major operating system variants and command-line interaction tools) is important not just for this class but also for your professional development. This activity has a couple of thought/research questions in Task 1, then tasks that are really just following a script (what we call "punch and run"). You can complete these last 3 tasks without fully understanding what you are doing and still receive full credit – but ideally the motivation here is to encourage you to spend time familiarizing yourself with your Linux, VM, and Docker environments and command-line tools before the class starts more intensive activities. The time you spend now will serve you soon in this course, and more importantly in your professional development.

TASK 1: Security Vulnerabilities (27 points)

Please answer the following in your submission. Prefix the section of your submission as "Task 1" and number each answer. A short paragraph will suffice (2-3 sentences) but please use complete sentences and spell and grammar check!

Question 1 (15 points): NIST (The National Institute of Standards and Technology) supports the National Vulnerability Database (NVD) at https://nvd.nist.gov. You may search the NVD for CVEs (Common Vulnerabilities and Exposures) at https://nvd.nist.gov/vuln/search. Use this tool to answer the following (use "Exact Match" under Keyword Search and "Last 3 Months" under Search Type)

1. Search "ubuntu". Answer:
   a. How many CVEs have been created so far for Ubuntu in the last 3 months?
   b. Pick any one CVE that interests you, and click on the details page (the Vuln ID). Summarize the CVE, its severity, and provide your thoughts on its significance.
2. Search "npm". Answer:
   a. How many CVEs were created so far for npm in 2024?
   b. Pick any one CVE with a High or Critical 3.1 severity that interests you, and click on the details page. Summarize the CVE, its severity, and provide your thoughts on its significance.
   c. Select one of the 3.1 Critical vulnerabilities. On its details page is a "Hyperlink" table. Follow the links, inspect the resulting pages and the information on them. From your reading, do you consider this CVE high severity, moderate severity, or low severity? Is there a fix or compensating action specified and easy to implement?

Question 2 (12 points): For each of C-I-A, give a separate example of each that have affected you (experienced a loss of it). At least one of the examples must be from a computing system (broadly defined) and at least one must be from a non-computing environment (i.e. the outside world).

Tasks 2-4 Preamble: The tasks specified next are "follow-me" activities intended to increase your familiarity with Linux, docker, and Wireshark. The tasks are written presuming you are running the SEED VM provided for the class. If you are presently unable to run the VMs (perhaps due to the M1 issue), it is possible to complete these activities using an alternate Linux environment (Task 2), docker on your host or a VM (Task 3), and Wireshark on your host. However, your screens won't match our exactly (especially when you do the networking-related steps), and you will need to transpose the information we provide you to your environment. Finally, you could also use a generic Ubuntu VM, and these you can get on Apple M1 silicon. Later labs will need the SEED VM.

Tasks 2-4 Setup: Start the SER335-Ubuntu20.04 Virtual machine and open a new terminal. In each task you will see yellow textboxes indicate screen captures you should take and copy into your submission.

If you are not running the SEED VM: Task 2 is generic Linux commands. You could do these on a Mac, WSL on Windows, a generic Linux VM (not SEED), or in an interactive docker container based on linux. Git bash however doesn't count.

TASK 2: Linux Walkthrough (18 points)

Present working directory

The **pwd** command prints the working directory (the current directory) from the root / directory.

```
[01/13/21]seed@VM:~$ pwd
/home/seed
[01/13/21]seed@VM:~$
```

Task 2.1: What is your current directory?

Make a new directory

Let's create a new Exercise1 directory. The **mkdir** command allows you to create new directories in the filesystem.

Syntax: mkdir file_name

Command: mkdir Exercise1

List files and directories

View the newly created directory using the **ls** command. The ls command lists the files in the current directory.

Command: ls

Task 2.2: List all your files and directories in current directory

NOTE: To list the files and folders in the current directory with a detailed listing use the -l (long) option

Command: ls -l

```
[01/13/21]seed@VM:~$ mkdir Exersice1
[01/13/21]seed@VM:~$
[01/13/21]seed@VM:~$ ls
Desktop    Downloads  Music     Public     Videos
Documents  Exersice1  Pictures  Templates  volumes
[01/13/21]seed@VM:~$
```

Change Directory

Go inside the **Exercise1** directory using the **cd** command. **cd** moves you to a new directory in the filesystem.

Syntax: cd file_name

Command: cd Exersice1

```
seed@VM:~$ cd Exersice1
seed@VM:~/Exersice1$
```

Create a new python file

Using the **vi** command, create a **helloworld.py** file. Vi is a screen-oriented text editor developed for Unix

Syntax: vi file_name

Command: vi helloworld.py

This opens a vi editor in text mode. Text mode enables us to perform administrative tasks such as saving files, executing commands, moving the cursor, cutting and pasting lines or words, and find/replace.

The vi editor is in **command mode** by default, type "**i**". It enables you to write your code. Add the following lines of code.

```
# This program prints Hello, world!
print('Hello, world!')
```

To save and exit vi, press ESC and type **':wq'**. Press ESC and type '**:q**!' to exit the editor without saving the changes.

View file contents

We can view the contents of helloworld.py using the **cat** command.

Syntax: cat file_name

Command: cat helloworld.py

Task 2.3: Display your program using 'cat' command

```
[01/13/21]seed@VM:~/Exersice1$ cat helloworld.py
# This program prints Hello, world!

print('Hello, world!')
```

Change the file permissions

Files and directories in Unix may have 3 types of permissions: read (`r'), write (`w'), and execute (`x'). Each permission may be `on' or `off' for each of 3 categories of users: the file or directory owner; other people in the same group as the owner; and all other. Let's provide execution permission to all users.

Syntax: chmod a+x file_name

Command: chmod a+x helloworld.py

Task 2.4: Change the permissions of your file and view the changes using 'ls -l'

Type **'ls -l'** command to view the permission of file or directory.

```
[01/13/21]seed@VM:~/Exersice1$ chmod a+x helloworld.py
[01/13/21]seed@VM:~/Exersice1$ ls -l
total 4
-rwxrwxr-x 1 seed seed 61 Jan 13 13:53 helloworld.py
[01/13/21]seed@VM:~/Exersice1$
```

Program Execution

Execute the helloworld program and look at the output.

```
[01/13/21]seed@VM:~/Exersice1$ python3 helloworld.py
Hello, world!
[01/13/21]seed@VM:~/Exersice1$
```

Task 2.5: Provide screenshot of successful execution of the program

Change User

The **su** command is used to become another user during a login session. Let's switch to the **root** user using the su command.

Syntax: su username

Command: su root

NOTE: If the root password is not set, follow the below instructions to change the root password and login in as root.

Change root password: sudo passwd root

Enter New Password: New password ==> dees

Switch to root user: su root

Task 2.6: Execute the above program with root user

TASK 3: Docker (25 points)

Note: This follow-me activity assumes you are running docker inside the SEED VM. You can do this task on your host and not in the VM, but the network setup and IP addresses will not be the same as we show below. You will have to understand your networking interfaces and what should appear. We are fine grading the task however you want to do it, though the preference again is to use the SER335 SEED VM if possible.

Docker Terminology

A **Docker image** is a read-only file that contains the source code, libraries, dependencies, tools, and other files needed to run an application.

Docker Hub repositories allow us to store and share container images with your team, customers, or the Docker community at large. Docker images are pushed to Docker Hub through the docker push command. A single Docker Hub repository can hold many Docker images. We can also pull the docker images into our system as per our requirement.

Docker Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a Compose file to configure your application's services. Then, using a single command, you create and start all the services from your configuration.

Playing with Alpine Docker

Alpine Docker Image: A minimal Docker image based on Alpine Linux with a full package index and only 5 MB in size!

Now that we have everything set up, we are going to run an alpine container on our system and get a taste of the how docker and docker compose works.

Docker Pull

The *pull* command fetches the image (Alpine Linux) from the Docker registry and saves it to our system.

Command: docker pull alpine

Docker Images

You can use the docker images command to see a list of all images on your system.

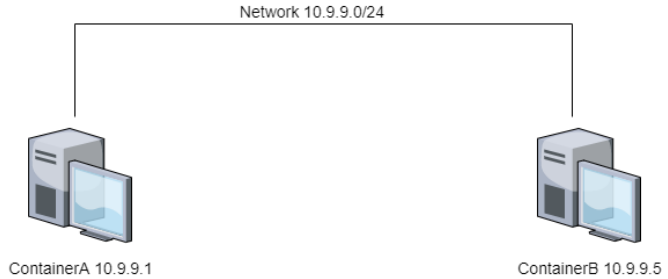
Command: docker images

```
[01/12/21]seed@VM:~$ docker images
REPOSITORY                    TAG                 IMAGE ID            CREATED             SIZE
alpine                        latest              389fef711851        3 weeks ago         5.58MB
```

Task 3.1: Use 'docker images' command to list all images

Environment Setup using Containers

We will use two machines (containers) that are connected to the same LAN



Environment Setup

```
version: "3"
services:
    ContainerA:
        image: alpine:latest
        container_name: ContainerA
        tty: true
        cap_add:
                - ALL
        privileged: true
        network_mode: host
    ContainerB:
        image: alpine:latest
        container_name: ContainerB
        tty: true
        cap_add:
                - ALL
        networks:
            alpine_net:
                ipv4_address: 10.9.9.5
networks:
    alpine_net:
        ipam:
            config:
                - subnet: 10.9.9.0/24
```

Define services in a Compose file

Create a file called docker-compose.yml in your current directory and paste the code to the left.

As we can see, docker-compose.yml specifies two containers ContainerA and ContainerB. Both use the alpine: latest image that we pulled earlier. The tty is set to true to enable an interactive shell to the user. cap_add property is set to ALL so that all the container capabilities are assigned.

In ContainerA, the privilege flag is set to true. This grants a docker container root capabilities i.e. the capabilities of the host machine. The network_mode is set to host so that containerA can see the packets among other containers. When a container is in the host mode, it sees all the host's network interfaces, and it even has the same IP addresses as the host. It is put in the same network namespace as the host VM.

For Container B a new network "alpine_net" is created with ipv4_address set to 10.9.9.5 and subnet 10.9.9.0/24

Build and run the containers:

Run command "**docker-compose up**" to build and run the container.

Command: docker-compose up

NOTE: You will need a new terminal window going forward below, as this compose command does not exit

Now it's time to see the running containers. The "**docker ps**" command shows you all containers currently running.

Command: docker ps

Task 3.2: Use 'docker ps' command to list the running containers

Inspect the networking between containers from within docker

Using the docker network command, inspect the available networks presently up.

Command: docker network ls

Task 3.3: Use 'docker network ls' to list the available networks

Go inside a container

We can go inside a ContainerA using the exec command.

Syntax: docker exec -it <container_name> /bin/sh

Command: docker exec -it ContainerA /bin/sh

Task 3.4: Use 'ls' to get a directory listing from within container A

Getting the network interface name:

When we use the provided Compose file to create containers, a new network is created to connect the VM and the containers. The IP prefix for this network is 10.9.9.0/24, which is specified in the docker-compose.yml file. The IP address assigned to ContainerA is 10.9.9.1. We need to find the name of the corresponding network interface on our VM because we need to use it in the Wireshark application. The interface name is the concatenation of 'br-' and the ID of the network created by Docker. When we use ifconfig to list network interfaces, we will see quite a few. Look for the IP address **10.9.9.1**.

$ **ifconfig**

```
br-a7c692d026c6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.9.1  netmask 255.255.255.0  broadcast 10.9.9.255
        inet6 fe80::42:95ff:fe21:92f3  prefixlen 64  scopeid 0x20<link>
        ether 02:42:95:21:92:f3  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 45  bytes 8234 (8.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

IP Address of **ContainerA** is **10.9.9.1** (same as host IP) so, what is IP address of ContainerB?

Open a new terminal and go inside(exec) ContainerB. Enter the command ifconfig to know the see IP address of ContainerB. It should be **10.9.9.5**

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:0A:09:09:05
          inet addr:10.9.9.5  Bcast:10.9.9.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:70 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:12904 (12.6 KiB)  TX bytes:0 (0.0 B)
```

Task 3.5: Type ifconfig and capture what your network interfaces and IP addresses are for ContainerA and ContainerB

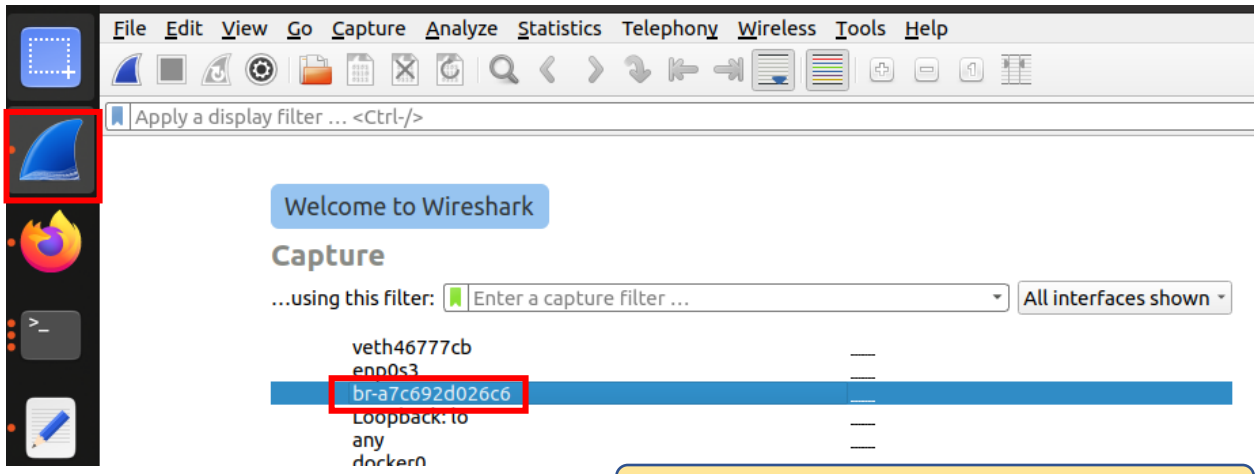
Note: *ifconfig* is a Unix utility for inspecting network interfaces. You have a corresponding command-line tool in DOS (Windows) named *ipconfig*.

TASK 4: Exploring the Wireshark Application (30 points)

Note: Again, if you are not using the VM you will need to transpose the IPs we are asking you to filter on to the corresponding ones you see. For example, ContainerA should share your host's IP address if you are running directly on your host and not in the VM (for example, 192.168.1.XXX on my local network at home).

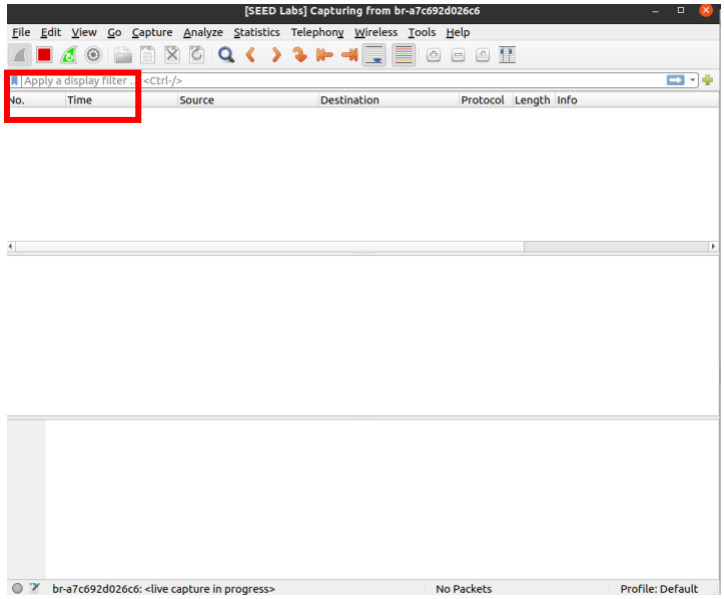
Wireshark is a free and open-source packet analyzer. It is used for network troubleshooting, analysis, software and communications protocol development, and education.

Open the Wireshark application within your VM and double-click on the interface found out in the previous step 3.5.



Task 4.1: Provide a screenshot of your Wireshark

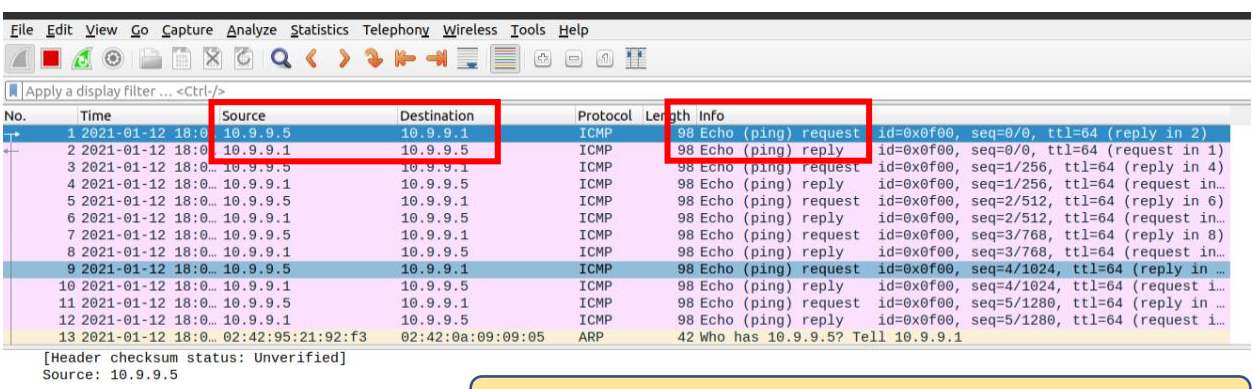
A new screen opens, indicating at the bottom that it is capturing all the packets of the given interface. You can restart or stop and start using the buttons indicated in the below figure.



From the ContainerB terminal execute **"ping 10.9.9.1".**

```
/ # ping 10.9.9.1
PING 10.9.9.1 (10.9.9.1): 56 data bytes
64 bytes from 10.9.9.1: seq=0 ttl=64 time=0.147 ms
64 bytes from 10.9.9.1: seq=1 ttl=64 time=0.086 ms
64 bytes from 10.9.9.1: seq=2 ttl=64 time=0.089 ms
64 bytes from 10.9.9.1: seq=3 ttl=64 time=0.086 ms
64 bytes from 10.9.9.1: seq=4 ttl=64 time=0.112 ms
```

So, what's happening here? We are pinging (contacting) ContainerA from ContainerB. With each ping, we are transmitting a pack with a Ping request from the container (10.9.9.5) to ContainerA (10.9.9.1) and receiving a ping reply from ContainerA (10.9.9.1) to ContainerB (10.9.9.5). This can be clearly seen in the Wireshark application.



Task 4.2: Provide screenshot of your Wireshark after ping

Filter packets

Looking at the screenshot above, you can see ICMP protocol packets, but you also at the bottom see ARP protocol packets. Next you should filter by packet protocol.

Task 4.3: Filter packets Wireshark displays by protocol ICMP and provide screenshot of your Wireshark. Then, switch to filter by protocol ARP and make a new screen capture.

Exit from a container:
Use **exit** command to come outside of the container.

Stop Docker Container:
Use docker stop

Syntax: docker stop container_id

Command: docker stop 55540dae38a0

Delete a container:

*Syntax***:** Docker rm container_id

Command: docker rm container 55540dae38a0

```
[01/12/21]seed@VM:~$ docker stop 55540dae38a0
55540dae38a0
[01/12/21]seed@VM:~$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
[01/12/21]seed@VM:~$
```

NOTE: When you stop a container you may restart it, but if you delete the container state is lost. The image(s) used to create the container will still be present, though these may be removed as well to conserve disk space (provided you are sure you are really done with them!).

Task 4.4: Provide a screenshot of your docker stop followed by a docker ps as shown above